

AfterBloom

Next Round Implementation Plan

From Alert Overload to One Clear Action

Team Mavericks | She Solves 3.0 | Health

This document lists each of the four challenges exactly as given on the challenge sheet, followed by our solution: what we build, how it works, what changes in the current AfterBloom prototype (database, API routes, screens and tests), and how we will demonstrate it. The last sections give the build plan for the day, the demo flow and a checklist.

*Note: table and route names are taken from our project report (for example **flags**, **alerts**, **audit_log**, **care_loops**, **consent_log**, **/api/alerts/emergency**). Match them to the exact names in the repository while building.*

The challenge brief (from the sheet)

CHALLENGE: FROM ALERT OVERLOAD TO ONE CLEAR ACTION

The existing AfterBloom prototype already supports red/amber/green guidance, EPDS screening, human escalation, clinician dashboards, scheduled screenings, family alerts with consent, audit logs, and professional review. The goal is to extend the existing AfterBloom prototype so that professionals can handle multiple related alerts efficiently without losing important information or treating every alert as a separate incident.

Our approach in one line: Signal, Case, One Action

All four challenges are solved by one pipeline added on top of the existing prototype. Each layer builds on the one before it.

Layer	What it does	Challenge
Signal	Every alert, from any part of the app, is saved in one standard shape and linked to related or duplicate alerts for the same patient.	1
Case	Related signals are combined into one patient case with a timeline showing what changed, when, and which signals triggered it.	2
One Action	Each case gets one priority and one recommended next action. Every action is audited, and family sharing goes through her existing consent switches.	3
Extras	Escalation ladder, consent request, shift handover, alert-load meter and patient transparency, all built on the same data.	4

The running example used throughout

Priya, day 9 after a C-section, with high blood pressure in pregnancy. In the current prototype one bad evening produces this:

Time	Source in the app	What fires today
Mon 7:10 pm	Daily check-in: BP 148/94 and headache	RED flag

Time	Source in the app	What fires today
Mon 7:30 pm	Symptom checker: "sar mein bahut dard aur dhundhla dikh raha hai"	RED flag via /api/alerts/emergency
Mon 10:30 pm	Care loop: no reply 2 hours after the follow-up question	Escalation flag
Tue 8:00 am	Check-in BP 152/96, so the two-raised-readings rule fires	AMBER pattern alert
Tue 12:30 pm	Daily cron: callback overdue	Re-flag
Earlier that week	EPDS score 11; partner screening: "not sleeping, withdrawn"	2 more items
Tue 9:00 am	Baby: yellow skin reported in the symptom checker	AMBER flag

Dr Rao sees **nine separate rows**. Clinically there are only **three concerns**: possible postpartum pre-eclampsia, a mood concern building over the week, and the baby's jaundice. Our target for the demo: **9 alerts become 3 cases, each with one clear next action, and no original alert is lost.**

Challenge 1: Related alerts for the same patient

CHALLENGE 1: IDENTIFY AND REPRESENT RELATED ALERTS FOR THE SAME PATIENT

Extend the existing prototype so that multiple alerts generated for the same patient can be identified as related or potentially duplicated, instead of being treated as completely separate incidents.

1.1 Turn every alert into one standard signal

Today about ten places in the app create flags, each in its own way. We add one function, **recordSignal()**, that every one of them calls. It saves a row in a single standard shape. The original rows in checkins, symptom_logs, epds_results and the rest stay exactly as they are, so **no information is lost**.

Field	Example	Purpose
mother_id	Priya	Same patient
subject	mother / baby	Baby signals form their own group
source	checkin, symptom_checker, bp_trend, epds, care_loop, partner_screen, circle_post, cron	Which channel raised it
code	SEVERE_HEADACHE, BP_RAISED, EPDS_PROBABLE, NO_LOOP_REPLY	Machine-readable, never free text
concern	HYPERTENSIVE	The key used for grouping (1.2)
severity	red / amber / info	Highest always wins
value	{"sys":148,"dia":94}	Structured numbers only
observed_at	timestamp	When it happened, not when it was saved
origin_table, origin_id	checkins, row id	Link back to the original record
parent_signal_id, relation, relation_reason	set by the matcher	The answer to Challenge 1

Signals never store what the mother typed, which keeps our existing promise that typed symptom text is never saved.

1.2 Give every symptom a clinical concern

A new field, **concern**, is added to the symptom catalogue (about 125 symptoms) and to every other signal code. The mapping lives in **clinical_config**, beside the triage levels, so a clinician can review and edit it from the admin page.

Concern	Signals that map to it
HYPERTENSIVE (pre-eclampsia)	Raised BP, BP trend, severe headache, blurred vision, face swelling
HAEMORRHAGE	Heavy bleeding, large clots, dizziness or fainting
INFECTION	Fever, chills, foul discharge, wound redness, painful red breast, burning urination
CLOT_RISK	Painful swollen calf, chest pain, breathlessness
MOOD	EPDS 10 or more, low mood in check-ins, not sleeping, partner screening concern
SELF_HARM	EPDS question 10, self-harm wording anywhere, postpartum psychosis signs
NEWBORN	The 47 baby symptoms (grouped per baby)
ENGAGEMENT	Missed check-ins, no care-loop reply, overdue callbacks

1.3 Linking rules (plain rules, no AI, fully explainable)

When a signal arrives, it is compared with that mother's recent signals and given exactly one relation. Rules run in this order:

#	Relation	Rule	Label shown to the professional
1	FOLLOW_UP	Created because of an earlier alert (care-loop no reply, overdue callback, re-flag). Linked through the loop or flag id it already carries.	Follow-up of: Headache + blurred vision (7:30 pm)
2	DUPLICATE	Same code, same source, within 12 hours	Possible duplicate ×2
3	CORROBORATES	Same code, different source, within 12 hours	Also reported in the symptom checker
4	RELATED	Same concern, within that concern's window	Same concern: blood pressure
5	POSSIBLY RELATED	A clinical cross-link from a small rule table (for example fever with wound redness, or poor sleep with EPDS 11)	Possibly related: confirm?
6	NEW	None of the above	New concern

Concern	Window (draft, for clinician sign-off)
Hypertensive	72 hours
Haemorrhage	48 hours
Infection	72 hours
Mood	14 days (matches the EPDS repeat-screen gap)
Self-harm	7 days

The sheet says *related or potentially duplicated*, so we use two confidence levels. Rules 1 to 4 link automatically. Rule 5 is only a suggestion: the professional taps **Confirm** or **Unlink**, and the decision is

written to the audit log. This mirrors the "Did you mean...?" pattern already used in our symptom checker.

1.4 Safety rules the grouping must never break

1. **Grouping hides noise, never danger.** A signal that raises severity (amber to red), opens a new concern, or is a self-harm signal always sends a fresh real-time alert, even inside an existing group. Only true duplicates and follow-ups are held back from re-notifying.
2. **Severity never goes down.** A group's severity is the highest severity of anything in it, matching our existing rule that answers can only raise the level.
3. **Nothing is deleted.** Duplicates are collapsed in the view with a count, but every row is kept and can be expanded.
4. **Consent still applies.** Matching runs on the server across all signals; the professional's view shows only what she may see. Red and self-harm signals are always visible, as today; mood and check-in details only while the mother is sharing.

1.5 Database: migration 007_alert_signals.sql

```
create table signals (  
  id uuid primary key default gen_random_uuid(),  
  mother_id uuid not null references mothers(id) on delete cascade,  
  subject text not null default 'mother',          -- 'mother' | 'baby'  
  source text not null, code text not null, concern text not null,  
  severity text not null,                          -- 'red' | 'amber' | 'info'  
  value jsonb default '{}',  
  observed_at timestamptz not null default now(),  
  origin_table text, origin_id uuid,  
  parent_signal_id uuid references signals(id),  
  relation text not null default 'NEW',  
  relation_reason text,  
  link_status text default 'auto',    -- 'auto' | 'suggested' | 'confirmed' | 'unlinked'  
  notified boolean default false,  
  case_id uuid,                       -- filled by Challenge 2  
  created_at timestamptz default now()  
);  
create index on signals (mother_id, concern, observed_at desc);  
create index on signals (mother_id, code, observed_at desc);  
alter table signals enable row level security;  -- policies mirror flags
```

Matching runs inside a Postgres function **record_signal(...)**, called through Supabase rpc. It starts with **pg_advisory_xact_lock(hashtext(mother_id::text))**, so two signals arriving in the same second (the check-in and the emergency route really do fire seconds apart) cannot both be marked NEW. This race-condition fix is worth mentioning to the judges.

1.6 Changes to the existing prototype

Where	Change
lib/signals/catalog.ts (new)	Code-to-concern map, windows and cross-link table, loaded from clinical_config with safe defaults
lib/signals/correlate.ts (new)	Pure function: new signal + recent signals in, relation + parent + reason out. Easy to unit test.
lib/signals/record.ts (new)	Calls the record_signal RPC and decides whether to notify (rule 1.4.1)
/api/alerts/emergency	Calls recordSignal instead of inserting a flag directly
/api/epds	Probable result becomes EPDS_PROBABLE; question 10 becomes EPDS_Q10 (SELF_HARM)

Where	Change
/api/loop	No reply, cannot get there and feel worse carry the parent signal id
/api/partner-screen	PARTNER_CONCERN under MOOD
Check-in save path	Danger signs and BP readings become signals
/api/cron/daily, /api/cron/loops	BP trend, overdue callback and no reply become FOLLOW_UP or RELATED signals, not fresh flags
/api/circles/post	Self-harm wording becomes SELFHARM_TEXT (still also goes to the moderator queue)
Professional dashboard	Alerts grouped under their concern with relation badges, a "Why linked?" line, and Confirm / Unlink on suggestions
ASHA dashboard	"Raised BP" warning reads from the grouped concern instead of raw rows

The old flags table keeps working (it follows the signals), so the current dashboard never breaks halfway through the work.

1.7 Tests to add

- Same code within 12 hours is DUPLICATE; after 12 hours it becomes RELATED or NEW.
- Same code from a different source is CORROBORATES.
- A no-reply signal is FOLLOW_UP and points to the correct parent.
- Amber to red inside a group still notifies; a self-harm signal always notifies, even as a duplicate.
- A group's severity never goes down.

1.8 Demo moment

A dev route `/api/dev/seed-storm` and a **Simulate alert storm** button insert Priya's evening with real timestamps. Show the old view (9 rows), then the grouped view: **"9 alerts, 3 concerns, every original alert one tap away."**

Challenge 2: One patient case and timeline

CHALLENGE 2: CONSOLIDATE RELATED ALERTS INTO ONE PATIENT CASE AND TIMELINE

Combine related alerts into a single patient case/timeline and clearly show what changed, when it changed, and which signals triggered the concern.

2.1 What a case is

A **case** is one clinical concern for one patient (or her baby), for example "Possible high blood pressure after birth". Every related signal from Challenge 1 is attached to the same case. The professional works on cases, not on individual alerts.

Situation	What happens to the case
Signal is NEW and no open case exists for that concern	A new case opens. This signal is marked as the trigger .
Signal is DUPLICATE, CORROBORATES, RELATED or FOLLOW_UP	Attached to the parent's case. Duplicates add to a count.

Situation	What happens to the case
Suggested link confirmed by the professional	The two cases are merged ; both concern tags are kept. Audited.
Professional unlinks a signal	The signal is split out into its own case. Audited.
New signal arrives within 7 days of a resolved case	The case reopens and is labelled "Reopened", instead of starting from zero.
No new signals and all actions done	Moves to Monitoring . Only a professional can mark it Resolved ; red and self-harm cases never close automatically.

2.2 Showing what changed, when, and what triggered it

The case page answers the three questions in the challenge directly, each in its own clearly labelled part of the screen.

Question	How the case page answers it
Which signals triggered the concern?	A Triggered by panel at the top lists the signal that opened the case and every signal that later raised its severity, for example: "Opened by: check-in BP 148/94 + headache, Mon 7:10 pm. Raised to red by: symptom checker, headache + blurred vision, 7:30 pm."
What changed?	A What changed strip shows differences, not just values: BP 148/94 to 152/96 (+4/+2); severity amber to red; EPDS 11 to 14 (+3); sleep average 5.5 to 3.8 hours; new concern added. Small sparklines for BP and EPDS sit in the case header.
When did it change?	A vertical timeline grouped by day. Each entry has a time, a source icon, a severity chip (colour plus icon plus word, for accessibility), and its relation. Duplicates collapse to "x2"; follow-ups sit indented under their parent.
What is new for me?	A Since you last looked marker, using the professional's last view in the audit log: "3 new events since 9:15 am", with those events highlighted.

2.3 Case summary sentence (templates, not AI)

Each case gets one plain summary sentence built from fixed templates filled with data, following our rule that AI never writes health content:

"3 signals in 13 hours point to raised blood pressure. BP rose from 148/94 to 152/96, with headache and blurred vision. No reply to the follow-up question since 8:30 pm."

2.4 What the mother sees

The mother never sees a case, a priority or a score, in line with our rule that worrying results reach a human first. Her home screen only shows a calm status card: **"Your care team is looking into this. They will contact you."**

2.5 Database: migration 008_cases.sql

```

create table cases (
  id uuid primary key default gen_random_uuid(),
  mother_id uuid not null references mothers(id) on delete cascade,
  subject text not null default 'mother',
  concerns text[] not null,          -- primary concern first; more after a merge
  title text not null,               -- from a template
  status text not null default 'open', -- open | monitoring | resolved
  severity_peak text not null, severity_current text not null,
  priority text, due_by timestampz,  -- filled by Challenge 3
  owner_pro_id uuid references pros(id),
  trigger_signal_id uuid references signals(id),
  signal_count int default 0,
  opened_at timestampz default now(), last_signal_at timestampz,
  resolved_at timestampz, reopened_count int default 0
);
create table case_events (          -- append-only timeline
  id bigserial primary key,
  case_id uuid not null references cases(id) on delete cascade,
  at timestampz not null default now(),
  type text not null,
  -- signal_added | severity_changed | priority_changed | status_changed | merged
  -- | split | link_confirmed | action_taken | family_notified | note
  actor_role text not null, actor_id uuid,
  detail jsonb default '{}', -- e.g. {"from":"amber","to":"red","signal_id":"..."}
);
alter table signals add constraint signals_case_fk
foreign key (case_id) references cases(id);
-- RLS: case_events is insert-only (no update/delete): the timeline cannot be rewritten

```

The **record_signal** function from Challenge 1 is extended: after deciding the relation, it opens or attaches to a case, updates severity and counts, and appends the matching case_events rows, all in the same transaction.

2.6 Changes to the existing prototype

Where	Change
record_signal (RPC)	Open, attach, reopen cases; append case_events; keep severity_peak and severity_current
GET /api/cases	Professional's list of cases (consent-filtered)
GET /api/cases/[id]	Case header, triggers, deltas, timeline. Every open is written to audit_log (existing view log).
POST /api/cases/[id]/merge, /split, /status	Professional merge, split, monitor, resolve. Each writes case_events and audit_log.
lib/cases/summary.ts (new)	Template sentences per concern, filled with data
lib/cases/deltas.ts (new)	Compares numeric series (BP, EPDS, sleep, mood) and returns what changed
/pro/cases/[id] page (new)	Header with sparklines, Triggered by, What changed, timeline with filters by source
Mother home screen	Neutral "care team is looking into this" card while a case is open
Supabase Realtime	Subscribe to cases and case_events so the case page updates live

2.7 Tests to add

- NEW opens a case; RELATED attaches to the same case; a signal 8 days after resolve opens a new case, at 5 days it reopens.
- Merge keeps every signal and both concerns; split moves only the chosen signal.

- Delta calculation for BP pairs and EPDS scores; severity_peak never decreases.

Challenge 3: One clear, priority-based workflow

CHALLENGE 3: CREATE ONE CLEAR, PRIORITY-BASED WORKFLOW FOR PROFESSIONAL ACTION

Extend the system so that the professional can see the patient's consolidated timeline, understand the signals that caused concern, identify the current priority, and determine the next appropriate action. The system should also maintain an audit trail recording the action taken. The workflow should respect the patient's existing consent and privacy settings when information is shared with family members.

3.1 The professional's new home: one Action Queue

The professional dashboard becomes an **Action Queue**: one card per case, never one per alert, sorted by priority and then by due time. The first card is always the one to handle next.

Part of the card	What it shows
Priority badge and timer	P1 Immediate, due in 42 min (turns red when overdue)
Title and concern chips	Possible high blood pressure after birth HYPERTENSIVE
Why this priority	Red signal unresolved; no reply to follow-up; pre-eclampsia history; day 9 postpartum
One big button	The single recommended next action, for example Call Priya now
Last action	Called 8:40 pm, not reached (Dr Rao)

A strip at the top reads: "**Today: 23 alerts consolidated into 6 cases. 1 P1, 2 P2.**"

3.2 How priority is decided (plain rules, explainable)

Priority	Rules (any one is enough)	Act within
P1 Immediate	Any SELF_HARM signal; postpartum psychosis signs; any unresolved red signal; red case with no reply to the care loop	1 hour
P2 Urgent	Amber getting worse (two ambers in a row, or a rising trend); red already handled but care not yet confirmed; EPDS 13 or more	4 hours / same day
P3 Soon	EPDS 10 to 12; a single amber; mood or sleep trend falling	48 hours
P4 Monitor	Only info signals; actions done and care confirmed	Next scheduled check

Modifiers can only raise priority by one level: no reply to a follow-up; callback overdue; high-risk history matching the concern (for example pre-eclampsia history for a hypertensive case); three or more independent sources agreeing; first 7 days after birth for hypertensive or haemorrhage cases.

Priority only goes down after a recorded action. It never drops just because time passed. Every rule that fired is stored as a reason, and the card shows these reasons under **Why this priority**. Weights and timers live in clinical_config for clinician sign-off.

3.3 One next action: the clinician playbook

For each concern and priority, a pre-written playbook (a data table, signed off by a clinician) gives exactly one recommended next action. Other actions stay available in a **More** menu.

Concern	P1	P2 / P3
HYPERTENSIVE	Call now. Direct her to hospital today; confirm she is going.	Call today. Arrange a BP recheck within 4 hours at the ANM or PHC.
SELF_HARM	Call now under the crisis protocol. If unreachable: emergency contact (only if she consented) and a Tele-MANAS handoff.	Not used: self-harm is always P1
MOOD	Call now (when combined with self-harm signals)	P2: callback within 24 hours and offer a session. P3: book counselling; repeat EPDS in 2 weeks.
HAEMORRHAGE	Call now. Send to hospital.	Call today; doctor review within 24 hours
INFECTION	Call now if fever with chills or foul discharge	Call today; doctor within 24 hours
NEWBORN	Call now. Baby to hospital or paediatrician.	Paediatric review within 24 hours
ENGAGEMENT	Not used alone	Assign an ASHA home visit

Secondary actions in the More menu: call, book a video session, assign an ASHA visit, refer to a psychiatrist, notify family (consent-checked, 3.5), add a note, monitor with a reason, hand over to a colleague, resolve with an outcome.

3.4 Recording the action: the audit trail

Every action needs a short outcome before the case can move on. One tap on the main button opens a small form:

Action	Outcome choices	What happens next
Call	Reached / Not reached / Wrong number	Not reached keeps the priority and sets a retry in 15 minutes for P1
Directed to hospital	Going now / Refused / Already there	Opens a care-loop check in 3 hours
Booked session	Date and time	Case moves to monitoring until the session
ASHA visit assigned	Due date	Task appears on the ASHA dashboard with reason only
Resolved	Outcome (seen by doctor, admitted, false alarm, other)	Status resolved; reopens if a related signal arrives within 7 days

Each action writes **three records in one transaction**: a case_actions row, a case_events row (it appears on the timeline), and an **audit_log** row recording who, what, when, which case, the outcome, and a snapshot of her consent settings at that moment. case_actions and audit_log are insert-only (no update or delete under RLS), so the trail cannot be edited. The mother can read a plain version: "Dr Rao called you at 8:40 pm."

3.5 Sharing with family: respecting her consent

This part answers the last sentence of the challenge, and it is enforced on the server, not only in the interface.

1. **Notify family** lists each family member with their current consent switches from consent_log. If Priya has not allowed safety alerts for a person, the button for that person is **disabled with the reason shown**: "Priya has not allowed alerts for Kamla (mother-in-law)."

2. Messages come **only from fixed neutral templates**, such as "Please call Priya and check on her today." The professional cannot type free text to family, so no diagnosis, score or case detail can leak.
3. The route `/api/cases/[id]/notify-family` checks consent again on the server and refuses if the switch is off, even if the interface was bypassed.
4. If consent is off but family help would be useful, the professional can send a **consent request** to Priya (see Challenge 4). Nothing is shared unless she says yes.
5. Every family message is logged with the consent snapshot, and Priya sees it: "Dr Rao sent Rohan a 'please call' message at 8:45 pm."
6. ASHA workers receive only a task and a reason ("Visit for BP check"), never the case details, as today.

3.6 Overdue actions

The existing loops cron (every 15 to 30 minutes) checks **due_by**. If a case passes its due time with no action, its priority goes up one level, a `priority_changed` event is written to the timeline, and the professional is notified again. This builds on the existing "overdue callback" rule instead of adding a new kind of alert.

3.7 Database: migration `009_case_actions.sql`

```
create table case_actions (
  id uuid primary key default gen_random_uuid(),
  case_id uuid not null references cases(id) on delete cascade,
  actor_id uuid not null, actor_role text not null,          -- pro | asha | admin
  action_type text not null,  -- call | hospital | book_session | asha_visit | refer
                                -- | notify_family | note | monitor | handover | resolve
  outcome text, note text,    -- short note, encrypted like EPDS answers
  next_due_at timestamptz,
  created_at timestamptz default now()
);
alter table audit_log add column if not exists case_id uuid,
  add column if not exists action text,
  add column if not exists consent_snapshot jsonb;
-- RLS: case_actions and audit_log are insert-only;
--      readable by the owning pro, admin, and (plain view) the mother
```

3.8 Changes to the existing prototype

Where	Change
<code>lib/cases/priority.ts</code> (new)	Pure function: case + signals + history in; priority, due_by and reasons out
<code>lib/cases/playbook.ts</code> (new)	Concern x priority to next action, from <code>clinical_config</code>
<code>GET /api/queue</code>	The professional's Action Queue, sorted by priority then due_by
<code>POST /api/cases/[id]/action</code>	Records an action and outcome; recomputes priority; writes <code>case_actions</code> , <code>case_events</code> and <code>audit_log</code> in one transaction
<code>POST /api/cases/[id]/notify-family</code>	Server-side consent check, template-only message, logged
<code>/api/cron/loops</code>	Overdue due_by raises priority one level and re-notifies
<code>/pro dashboard</code>	Replaced by the Action Queue; old patient list kept as a second tab
<code>/pro/cases/[id]</code>	Right-hand action panel: next action, outcome form, family panel with consent state, history

Where	Change
ASHA dashboard	Shows assigned visit tasks created from cases
Mother Privacy page	"What my care team did" list next to the existing view log

3.9 Tests to add

- Self-harm is always P1; priority never drops without a recorded action; overdue raises one level.
- Each modifier raises by exactly one level and never lowers.
- notify-family refuses when consent is off and succeeds when on; the message body is always a template.
- An action writes exactly one case_actions, one case_events and one audit_log row.

Challenge 4: Innovation beyond the requirements

CHALLENGE 4: INNOVATE BEYOND THE GIVEN REQUIREMENTS USING THE EXISTING AFTERBLOOM PROTOTYPE

Teams may introduce their own innovative feature, improvement, or intelligent workflow that helps professionals manage multiple patient alerts more effectively while preserving important information, privacy, consent, and auditability. The innovation must build upon or enhance the existing AfterBloom prototype. Innovative additions that demonstrate strong problem-solving, creativity, usability, workflow design, privacy awareness, or technical implementation can earn extra points.

Five additions, listed in the order we build them. Each reuses the signals, cases and actions from Challenges 1 to 3, and each maps to a scoring area named on the sheet.

#	Innovation	Scores on
4A	Escalation ladder with backup professional and ASHA	Workflow design, problem-solving
4B	"Ask her first": consent request from professional to mother	Privacy awareness, consent
4C	Alert-load meter for the hospital	Technical implementation, measurable impact
4D	Shift handover and "since you last looked" digest	Usability, preserving information
4E	Patient transparency: "What my care team did"	Auditability, trust

4A. Escalation ladder (no P1 case is ever left waiting)

If a P1 case is not acknowledged in time, it climbs a ladder automatically. Each step is a case_event and an audit_log row.

Step	When	Who is notified
1	At creation	Assigned professional (push + in-app)
2	15 minutes unacknowledged	Backup professional on duty (new field: backup_pro_id, or the hospital's on-call list)
3	30 minutes	The hospital admin desk, plus an ASHA visit task if she has an assigned ASHA
4	Any time	The mother has already seen the crisis screen with 112 and Tele-MANAS, which never depends on this ladder

Acknowledge is a single tap that stops the ladder and records who took ownership. Built inside /api/cron/loops plus an acknowledged_at field on cases.

4B. "Ask her first": consent request

When family help would be useful but Priya has not allowed it, the professional does not override her choice. They tap **Ask Priya**. She receives a neutral notification: *"Your care team would like to let Rohan know to check on you today. Allow once / Always / No."*

- **Allow once** sends one template message and changes nothing else.
- **Always** turns on the switch in her existing Family circle settings (recorded in consent_log).
- **No** is final for that case; the professional sees "Declined" and the decision is audited.

New table **consent_requests** (case_id, family_member_id, requested_by, status, answered_at). This turns consent from a fixed setting into a live, respectful conversation, which is unusual in health apps and easy to show in the demo.

4C. Alert-load meter

A new panel on the admin page, computed from signals, cases and case_actions, showing anonymous totals only:

Measure	Example
Alerts received vs cases created	214 alerts, 41 cases (81% less noise)
Duplicates collapsed	62
Median time from P1 creation to first action	11 minutes
Cases handled within their due time	94%
Cases by concern	Hypertensive 9, Mood 17, Infection 6, Newborn 7, other 2

This gives judges and hospital partners a number that proves the challenge is solved, and supports our hospital business model.

4D. Shift handover and "since you last looked"

- **Since you last looked:** every case shows how many new events arrived after the professional's last view, using the existing view log.
- **Handover:** one button builds a template-based handover note for the next professional: open cases by priority, last action on each, what is due next. Reassigning cases is audited, and the patient sees that her care moved to Dr Mehta.
- Nothing is lost between shifts, which is the core of the challenge: *without losing important information*.

4E. Patient transparency

A calm list in the mother's Privacy page: *"Dr Rao looked at your record (8:35 pm). Dr Rao called you (8:40 pm). Dr Rao asked to inform Rohan; you allowed once (8:44 pm)."* No scores, no priority, no case titles. It builds trust and makes the audit trail useful to the person it protects.

Optional, if time remains

- **What-if preview for clinical settings:** before an admin changes a grouping window (for example 72 to 48 hours), show how many current cases would split or merge.

- **Bulk acknowledge duplicates:** one tap acknowledges all duplicate amber signals in a case. Disabled for red and self-harm.

Safety, privacy and audit across all four

Principle	How it holds in the new system
Grouping never hides danger	Severity increases, new concerns and self-harm always re-notify, inside or outside a case
Severity and priority only go up automatically	Down only after a recorded professional action
Nothing is lost	Signals and timeline events are never deleted; duplicates are collapsed, not removed; merges and splits are reversible and audited
AI never writes advice	Linking, priority, summaries and next actions are all rules and templates, signed off in clinical_config
Consent first	Family sharing is checked on the server against consent_log, uses templates only, and can be requested but never overridden
Everything is auditable	Every view, link decision, action, family message and consent request is in the insert-only audit_log
Mother is never alarmed	She sees no cases, scores or priorities; only a calm status and a plain list of what her care team did
Crisis help is never gated	The crisis screen with 112 and Tele-MANAS works exactly as before, independent of the new workflow

Build plan for the day (about 7 hours, 3 accounts)

Step 0: agree the contract first (20 minutes, all together)

- Fix the column names of signals, cases, case_events and case_actions exactly as in this document. These are shared by all three accounts.
- Fix migration numbers in advance: 007 signals, 008 cases, 009 case_actions, 010 innovation tables. This avoids merge conflicts.
- Each account works on its own branch and its own folders; merge into main every 90 minutes and redeploy to Vercel.

Who builds what

Time	Account 1: database and engine	Account 2: routes and workflow	Account 3: screens and demo
0:20 to 1:50	Migration 007, record_signal RPC with lock, correlate.ts and tests	Route every alert producer through recordSignal	Seed-storm route and button; grouped alerts view with badges
1:50 to 3:20	Migration 008, case open/attach/reopen in RPC, deltas.ts, summary.ts	/api/cases, merge, split, status	Case page: header, Triggered by, What changed, timeline
3:20 to 5:00	Migration 009, priority.ts, playbook.ts and tests	/api/queue, /action, /notify-family with consent check, cron overdue	Action Queue, action panel, outcome form, family panel

Time	Account 1: database and engine	Account 2: routes and workflow	Account 3: screens and demo
5:00 to 6:15	4C alert-load meter queries; 4A ladder in cron	4B consent requests; 4D handover	4C admin panel, 4B mother prompt, 4E transparency list
6:15 to 7:00	Run all tests, final migration on production	Fix bugs found in the run-through	Two full demo rehearsals

If time runs short, the safe cut order is: optional items first, then 4D, then 4E. Never cut Challenges 1 to 3 or the consent check.

Demo flow for the judges (about 4 minutes)

1. Open Dr Rao's dashboard in the old patient-list tab. Press **Simulate alert storm**. Nine separate rows appear. "This is alert overload."
2. Switch to the **Action Queue**: three cards. "Nine alerts became three cases, and nothing was thrown away." Expand a case to show the duplicates and follow-ups. (Challenge 1)
3. Open the blood pressure case. Show **Triggered by, What changed** (148/94 to 152/96, amber to red) and the timeline. (Challenge 2)
4. Point at **P1, due in 42 minutes** and **Why this priority**. Tap **Call Priya now**, record "Reached, going to hospital". The case drops to monitoring and the action appears on the timeline. (Challenge 3)
5. Tap **Notify family**: Rohan is allowed, the mother-in-law is greyed out with the reason. Send Rohan the template message. Open the audit log to show the record with the consent snapshot. (Challenge 3)
6. Tap **Ask Priya** for the mother-in-law. Switch to Priya's phone, answer "No". The professional sees "Declined". (Challenge 4B)
7. Show Priya's "What my care team did" list, then the admin **alert-load meter**. Leave a P1 unacknowledged to show the ladder notifying the backup. (Challenge 4)

Done-when checklist

Challenge	Done when
1	Every alert producer goes through recordSignal; duplicates, corroborations, follow-ups and related signals are labelled with a reason; suggested links can be confirmed or unlinked; safety re-notify rules pass their tests
2	Related signals form one case; the case page shows triggers, what changed and a timeline; merge, split and reopen work and are audited; the mother sees only a calm status
3	One Action Queue sorted by priority; every case shows why its priority and one next action; every action writes case_actions, case_events and audit_log; family messages are consent-checked on the server and template-only
4	Escalation ladder, consent request, alert-load meter, handover and patient transparency all work in the demo
All	All new tests pass with the original nine; TypeScript has no errors; production build succeeds; the seeded demo runs end to end twice